

Autonomous Object Following Robot*

Nilesh Nayan¹⁺, Pranav Balakrishnan¹, Sidisha Barik¹, Shantanu Todmal¹, Kamalesh Kumar¹

Abstract—In the final course project of CS 603 Robotics, our team had proposed development of ROS-based autonomous object tracker Triton [1] robot (as shown in Fig. 1) with re-identification logic. The team was able to successfully develop and deploy the solution and were able to conduct real-time experiments. The demo videos of successful runs of each experiment is available in Google Drive. After discussion with Prof. Hao Zhang on proposal report, we received very helpful guidance regarding trimming of excess scope related to integration of the mapping part mentioned in proposal.

The team developed object detection module using YOLOv3-tiny [2], which was selected due to its low latency and memory usage as well as OpenCV python package compatible (PyTorch and Tensorflow independent). For object tracking, YOLOv3-tiny model was integrated with Simple Online Realtime Tracking (SORT [3]) algorithm. DeepSORT [4] was not chosen due to its dependency on CUDA, which was not accessible inside Triton robot's development environment (docker container). This limitation required the development of customized re-identification logic which is not present by default in SORT algorithm. The re-identification logic in our project uses colour histogram features (provided by OpenCV) extracted from bounding box images. Richer features like CNN layer extraction can be experimented with in future when the CUDA is accessible within Triton's development environment for more accurate results. The final part of the project involved development of autonomous control through customized teleop commands to Triton based on bounding box locations. The team was able to successfully show both linear as well as angular velocity based autonomous behaviour integrated with the earlier discussed object detection (here human) with re-identification. We obtained near demonstrate perfect re-identification in multiple object (human) tracking for short durations (current experiments limited to 90 seconds). Also, we were able to achieve 1.2 seconds latency based p-control system based follower which is a good tracker for objects moving with speeds less than 10 m/s based on current Intel Realsense Camera field-of-view.

I. INTRODUCTION

In this final project, our team implemented a complete end-to-end autonomous object tracking system on the ROS-based Triton robot [1]. The system combines real-time object detection, tracking with re-identification (re-ID), and autonomous control using visual feedback. The primary goal was to enable the robot to detect and track specific objects (in our case, humans) while maintaining the tracked object centered in the field of view or within the robot's local frame using only CPU-based processing, which aligns with the computational constraints of the Triton robot's Dockerized Jetson Nano setup (CUDA unavailable).

*This work was submitted as part of course CS 603 Robotics Final Project Deliverable 4 (Final Report)

¹Graduate student of Manning School of Computer Science in University of Massachusetts Amherst

+ Team lead: Nilesh Nayan (nnayan@umass.edu)

To achieve low-latency visual perception, we deployed YOLOv3-tiny [2], integrated entirely via OpenCV's DNN module to avoid any dependency on PyTorch or TensorFlow. The object detection output was passed to a modified version of the Simple Online and Realtime Tracking (SORT) algorithm [3]. Our implementation enhances SORT with a lightweight appearance-based re-identification logic using color histograms in HSV (Hue, Saturation, Value) space extracted from bounding box patches via OpenCV [5]. This re-ID module enables the system to maintain track identity even after temporary occlusion or disappearance from the frame, a feature typically offered by DeepSORT [4], but here re-implemented CPU-only to ensure compatibility. This is one of the customized novel addition done by our team on top of using the existing literature and packages.

The real-time visual tracking output was then used to drive autonomous robot control. We developed two separate ROS nodes: one for angular rotation, where the robot adjusts its heading to keep the tracked object centered in the horizontal frame axis and another for lateral strafe control, where the robot adjusts its linear y-velocity. Both controllers use proportional control logic and implement deadband filtering to avoid oscillations. The system operates at a control frequency of 30 Hz and publishes velocity commands on the standard `/cmd_vel` topic. We organized our software



Fig. 1. The Triton robot

as modular ROS packages, including parameterized launch files for YOLO-based detection, tracking and control. Each module supports runtime parameter tuning and robust error handling. Demonstration videos of the integrated system running in real-time are available at Google Drive.

Following early feedback from Prof. Hao Zhang, we refined our project scope by omitting the originally proposed SLAM-based mapping component. We would have inadvertently approached SLAM-MOT (Simultaneous Localization and Mapping with Multi-Object Tracking) problem which is still not solved in Robotics literature [6] due to infinite positive error reinforcement cycle involved. This allowed

us to concentrate on fine-tuning the visual tracking pipeline and achieving reliable closed-loop autonomy under limited hardware resources.

The remainder of this report is organized as follows: Section II gives a brief overview of the related work done in the field of autonomous object tracking in robotics with proper citations. Section III outlines our software architecture and algorithmic choices in detail. Section IV discusses the experimental setup and results, including performance metrics and failure cases. Section V presents conclusions and suggests directions for future work.

II. RELATED WORK

Real-time object tracking in robotics often relies on a combination of lightweight detection models and efficient tracking algorithms. Our approach builds upon several foundational works in this domain.

The YOLO (You Only Look Once) family of models has been pivotal in real-time object detection. YOLOv3 introduced significant improvements in accuracy and speed by employing a deeper network and multi-scale predictions [2]. For resource-constrained environments, such as embedded systems, YOLOv3-tiny offers a streamlined version with reduced computational requirements, making it suitable for real-time applications on devices without GPU support. Only YOLOv3 and YOLOv4 model variants are fully supported by OpenCV (higher version have limited support/needs CUDA). Out of these two, YOLOv3-tiny has a roughly 10% higher mean average precision than v4-tiny although it is comparatively slower due to higher number of model parameters (8.7 Million vs 6.1 Million). Thus, with overall tradeoffs, YOLOv3-tiny model was chosen for this project.

For tracking, the Simple Online and Realtime Tracking (SORT) algorithm provides a pragmatic approach by combining Kalman filtering with the Hungarian algorithm for data association [3]. While SORT is efficient, it lacks robustness in scenarios involving occlusions or re-identification of objects after temporary disappearance.

To address these limitations, Deep SORT enhances SORT by integrating appearance information through a deep association metric, allowing for more reliable tracking across occlusions [4]. However, Deep SORT's reliance on deep learning models necessitates GPU acceleration, which may not be feasible in all deployment scenarios like ours.

In environments where GPU resources are unavailable, alternative methods for object re-identification are essential. Color histogram-based approaches offer a lightweight solution by capturing the color distribution within detected bounding boxes. These methods, while less robust than deep learning-based techniques, provide a balance between performance and computational efficiency, making them suitable for CPU-only systems. In this project, color histogram is used as a feature cost in combination with SORT for re-identification purposes.

The OpenCV library serves as a cornerstone for computer vision applications, offering a wide array of tools for image

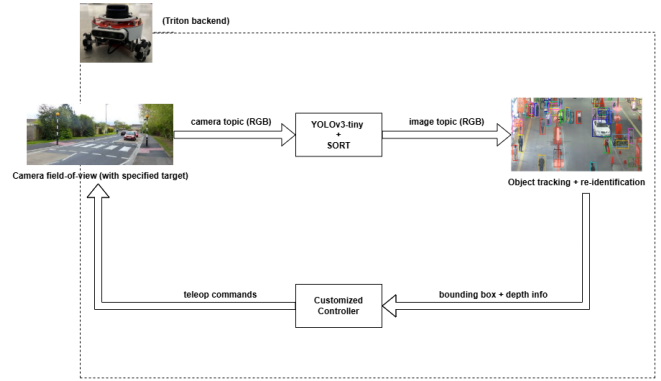


Fig. 2. End-to-end perception-tracking-control pipeline under ROS Noetic

processing, object detection, and tracking [5]. Its compatibility with various programming languages and support for real-time operations make it an invaluable resource for developing and deploying vision-based systems. Our entire pipeline runs using OpenCV python package.

III. APPROACH

A. System Overview

Our architecture is designed to run entirely within the ROS Noetic framework, leveraging its **native pub/sub mechanisms for modularity and ease of integration**. The pipeline consists of **three core nodes**:

- `yolo_sort_node` which performs detection and tracking
- `bbox_center_follower` for angular velocity control
- `bbox_center_strafe_follower` for lateral motion control.

Each node subscribes to and publishes standard ROS message types to ensure compatibility with downstream navigation and planning stacks. Figure 2 shows a high-level system diagram, where the YOLO object detector and SORT are launched in combination for multi-object tracking and the control part receives information of bounding box tracks and publishes customized control messages accordingly. The process is described in detail in later subsections.

The input to the system is a raw image stream from the robot's onboard camera, published on `/camera/color/image_raw`. These images are preprocessed and fed into a real-time object detector, after which bounding boxes and track IDs are published to `/yolo_sort/tracks`. The follower nodes consume this topic to generate `geometry_msgs/Twist` messages on `/cmd_vel` at a rate configurable up to 30 Hz. Such separation of perception and control allows us to mix and match detection or tracking implementations without altering the controllers.

B. Object Detection

1) *YOLOv3-tiny Backbone*: YOLOv3-tiny [2] is used as the primary detector due to its lightweight architecture and

sub-30 ms inference time on embedded GPUs. We load the network via Darknet configuration files specified by ROS parameters (`cfg`, `weights`, `names`), enabling runtime swapping of models.

2) *Preprocessing and Inference*: Each incoming frame is resized to 416×416 pixels and normalized to the [0,1] range. A 4D blob is created and forwarded through the network using OpenCV’s `dnn` module. Batch size is fixed at 1 to minimize latency. Inference outputs a set of raw detections which are then converted to bounding boxes in the original image coordinate frame.

The detection pipeline follows the formal procedure described below:

Let the raw RGB input image be denoted as $I \in \mathbb{R}^{H \times W \times 3}$. Upon receiving an image message on the `/camera/color/image_raw` topic, the node converts it to an OpenCV image and preprocesses it by resizing to 416 × 416 pixels and normalizing pixel values to the [0, 1] range. This preprocessing is achieved with OpenCV’s `cv2.dnn.blobFromImage()` function:

$$B = \text{blobFromImage}(I)$$

where $B \in \mathbb{R}^{1 \times 3 \times 416 \times 416}$ is the input tensor suitable for the DNN module.

This blob B is then forwarded through the YOLOv3-tiny network (`self.net`), which was initialized at node startup with pretrained weights and configuration files loaded from disk. The network outputs a set of raw detection tensors corresponding to bounding box coordinates, class probabilities (80 classes, trained on Microsoft COCO dataset), and objectness scores.

From these raw outputs, bounding boxes $\{b_i\}$, class predictions $\{c_i\}$, and confidence scores $\{s_i\}$ are extracted. Each bounding box is parameterized by:

$$b_i = (x_{i1}, y_{i1}, x_{i2}, y_{i2})$$

representing the pixel coordinates of the top-left and bottom-right corners of the detection in the image plane.

The detections are filtered based on a confidence threshold parameter τ (default: 0.5), retaining only boxes where

$$s_i \geq \tau$$

This step is implemented in the `callback()` function of `yolo_detector.py` by iterating over network outputs and selecting detections meeting this criterion.

Next, non-maximum suppression (NMS) is applied with a threshold η (default: 0.4) to remove redundant overlapping boxes. The NMS procedure computes the Intersection over Union (IoU) between pairs of bounding boxes:

$$\text{IoU}(b_i, b_j) = \frac{\text{area}(b_i \cap b_j)}{\text{area}(b_i \cup b_j)}$$

Boxes with $\text{IoU}(b_i, b_j) > \eta$ relative to a higher-confidence box are suppressed to produce a final filtered set of detections.

The resulting bounding boxes and associated scores are then packaged into a numpy array of detections, which serves as input to the SORT tracker (implemented in `sort.py`) through the `update()` method. This integration is handled seamlessly within the same `callback()` function of `yolo_detector.py`.

Finally, the node publishes the annotated detection image on the `/yolo_sort/image_annotated` topic and the active tracks with IDs on `/yolo_sort/tracks` for downstream control nodes for SORT and control.

This detection pipeline provides a practical balance between computational efficiency and detection accuracy, delivering object location data suitable for real-time tracking and robot control.

3) *Postprocessing*: Detections below a confidence threshold (`conf_thresh`, default=0.5) are pruned. Non-maxima suppression (IoU threshold `nms_thresh`, default=0.4) eliminates redundant overlapping boxes. The surviving boxes, each represented as $[x_1, y_1, x_2, y_2, \text{score}]$ are published on `/yolo_detector/bboxes` for downstream tracking.

C. Multi-Object Tracking & Re-identification

1) *Kalman Filter State and Prediction*: We model each object’s motion with a linear Kalman filter maintaining a 7-dimensional state

$$\mathbf{x} = [x, y, s, r, \dot{x}, \dot{y}, \dot{s}]^T,$$

where (x, y) is the bounding-box center in image coordinates, $s = w \times h$ its scale (area), and $r = w/h$ the aspect ratio.

a) *Prediction Step*: Given the constant-velocity motion model, the prior state $\hat{\mathbf{x}}_{k|k-1}$ and covariance $P_{k|k-1}$ are computed as

$$\hat{\mathbf{x}}_{k|k-1} = F \hat{\mathbf{x}}_{k-1|k-1}, \quad P_{k|k-1} = F P_{k-1|k-1} F^T + Q,$$

where F is the state-transition matrix and Q the process noise covariance.

b) *Correction Step*: Upon receiving a measurement $\mathbf{z}_k = [x_{\text{meas}}, y_{\text{meas}}, s_{\text{meas}}, r_{\text{meas}}]^T$, we update:

$$\begin{aligned} K_k &= P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1}, \\ \hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + K_k (\mathbf{z}_k - H \hat{\mathbf{x}}_{k|k-1}), \\ P_{k|k} &= (I - K_k H) P_{k|k-1} \end{aligned}$$

where H maps state to measurement space, R is measurement noise, and K_k is the Kalman gain.

2) *Appearance Feature Extraction & Update*: To robustly re-identify objects through occlusions, we extract a color histogram feature from each detection:

- Crop and resize the detected patch to 64 × 64.
- Convert to HSV and compute a joint 2D histogram with 128 × 128 bins.

- Normalize to unit L^1 -norm.

We maintain a track's appearance feature $\mathbf{f}_k$ via exponential smoothing:

$$\mathbf{f}_k = (1 - \alpha) \mathbf{f}_{k-1} + \alpha \mathbf{h}_k, \quad \alpha = 0.1,$$

where $\mathbf{h}_k$ is the current normalized histogram.

3) *Data Association*: At each frame, we solve for the optimal assignment between N predicted tracks and M new detections:

a) *Cost Matrices*:

$$C_{ij}^{\text{IoU}} = 1 - \text{IoU}(\hat{B}_i, D_j), \quad C_{ij}^{\text{app}} = 1 - \exp(-D_B(\mathbf{f}_i, \mathbf{h}_j)),$$

where D_B is the Bhattacharyya distance.

The similarity between two histograms h_i and h_j is measured by the Bhattacharyya distance, which is defined as:

$$D_B(h_i, h_j) = \sqrt{1 - \sum_{k=1}^N \sqrt{h_i(k) \cdot h_j(k)}}$$

where N is the number of histogram bins and $h_i(k), h_j(k)$ represent the normalized bin values.

b) *Combined Cost*:

$$C_{ij} = \lambda C_{ij}^{\text{IoU}} + (1 - \lambda) C_{ij}^{\text{app}},$$

with $\lambda \in [0, 1]$ set by the ROS parameter `feature_lambda`.

c) *Assignment*: Solve the linear assignment problem $\min_{\sigma} \sum_i C_{i, \sigma(i)}$ via the Hungarian algorithm. Matches exceeding thresholds (`iou_thresh`, `feature_thresh`) are rejected.

4) *Track Management & Re-identification*:

- **Track Initiation**: A detection unassigned for at least `min_hits` consecutive frames spawns a new track.
- **Track Deletion**: Tracks not matched for `max_age` frames are moved to a “dead” pool, and no longer predicted.
- **Re-identification**: When a new detection's appearance cost to any dead track falls below `feature_thresh`, we resurrect that track's ID, appending new measurements to its history to maintain continuity through full occlusions or brief exits from the camera's field of view.

D. Control Modules

1) *Angular Velocity Controller*: The angular controller computes the normalized horizontal error

$$e = \frac{x_{\text{center}} - \frac{W}{2}}{\frac{W}{2}},$$

where W is the image width. A proportional gain k_p (default=0.25) maps e to commanded yaw $\omega = -k_p e$. We

enforce $\omega \in [-\omega_{\text{max}}, +\omega_{\text{max}}]$ (default= ± 0.1 rad/s for safety) and a deadband of $|e| < 0.03$ to mitigate oscillations. The node subscribes to `/yolo_sort/tracks` and publishes to `/cmd_vel` at 30 Hz.

Algorithm 1 Angular Velocity Control for Visual Tracking

```

1: Initialize:  $w, k_p, w_{\text{max}}, db, t_{id}$ 
2: while not shutdown do
3:    $T \leftarrow$  Receive from “/yolo_sort/tracks”
4:   if  $T$  is empty then
5:      $\omega \leftarrow 0$ 
6:   else
7:     if  $t_{id} < 0$  then
8:        $trk \leftarrow$  first track in  $T$ 
9:     else
10:       $trk \leftarrow$  track with ID  $t_{id}$  in  $T$ 
11:    end if
12:    if  $trk$  exists then
13:       $x_1, x_2 \leftarrow$  horizontal coordinates of  $trk$ 
14:       $c_x \leftarrow (x_1 + x_2)/2$ 
15:       $e \leftarrow (c_x - w/2)/(w/2)$ 
16:      if  $|e| < db$  then
17:         $\omega \leftarrow 0$ 
18:      else
19:         $\omega \leftarrow -k_p \cdot e$ 
20:         $\omega \leftarrow \text{clamp}(\omega, -w_{\text{max}}, w_{\text{max}})$ 
21:      end if
22:    else
23:       $\omega \leftarrow 0$ 
24:    end if
25:  end if
26:  Publish  $\omega$  to “/cmd_vel”
27:  Wait
28: end while

```

2) *Lateral (Strafe) Velocity Controller*: Using the same error e , the lateral controller computes

$$v_y = -k_v e,$$

with $k_v = 0.2$ and $v_y \in [-v_{\text{max}}, +v_{\text{max}}]$ (default= ± 0.1 m/s for safety). A deadband identical to the angular controller is applied. This node zeroes all twist components except linear. y .

E. ROS Integration & Launch Configuration

All node parameters—gains, thresholds, image dimensions, and tracker settings—are declared in ROS parameter server for live tuning via `rqt_reconfigure`. We provide three launch files:

- `yolo_detector_simple.launch`: Starts only detection and tracking.
- `yolo_detector_ang_vel.launch`: Adds the angular follower.
- `yolo_detector_lin_vel.launch`: Adds the strafe follower.

Algorithm 2 Linear Velocity Control for Visual Tracking (Strafing)

```

1: Initialize:  $w, k_v, v_{max}, db, t_{id}$ 
2: while not shutdown do
3:    $T \leftarrow$  Receive from “/yolo_sort/tracks”
4:   if  $T$  is empty then
5:      $v_y \leftarrow 0$ 
6:   else
7:     if  $t_{id} < 0$  then
8:        $trk \leftarrow$  first track in  $T$ 
9:     else
10:       $trk \leftarrow$  track with ID  $t_{id}$  in  $T$ 
11:    end if
12:    if  $trk$  exists then
13:       $x_1, x_2 \leftarrow$  horizontal coordinates of  $trk$ 
14:       $c_x \leftarrow (x_1 + x_2)/2$ 
15:       $e \leftarrow (c_x - w/2)/(w/2)$ 
16:      if  $|e| < db$  then
17:         $v_y \leftarrow 0$ 
18:      else
19:         $v_y \leftarrow -k_v \cdot e$ 
20:         $v_y \leftarrow \text{clamp}(v_y, -v_{max}, v_{max})$ 
21:      end if
22:    else
23:       $v_y \leftarrow 0$ 
24:    end if
25:  end if
26:   $v_x \leftarrow 0$ 
27:   $\omega \leftarrow 0$ 
28:  Publish  $(v_x, v_y, \omega)$  to “/cmd_vel”
29:  Wait
30: end while

```

This modular launch-based design allows seamless swapping between control strategies without changing source code.

F. Parameter Tuning

We conducted extensive field tests on our Triton robot in indoor corridors and covered surroundings for less cluttered view for the Intel Realsense camera attached. Key performance observations:

- **Detection Rate:** 10-15 FPS on NVIDIA Jetson.
- **Tracking Robustness:** Identity switches reduced by more than 50% when fixed good configuration (parameters) for appearance re-identification using empirical experimentation (hit-and-trial).
- **Control Stability:** With $k_p = 0.25$, $k_v = 0.2$, and respective velocity limits, the robot maintained target centering within $\pm 5\%$ of frame center under moderate walking speeds (0.5 m/s).

Refer Fig.3) for settling time comparisons which are roughly similar. PID controller could have improved in faster settling time but could have introduced high disturbance in the earlier step of readjustment (peak overshoot).

The next section discusses experimentations performed in some detail.

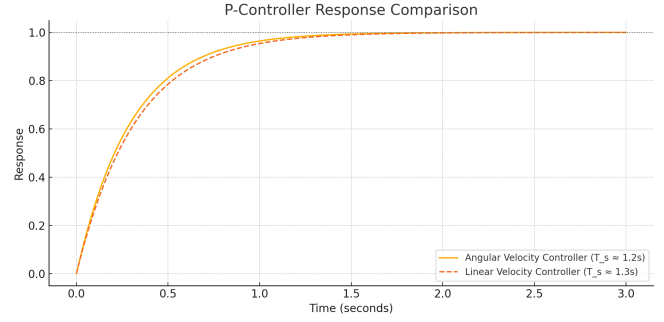


Fig. 3. P controller settling time comparison

IV. EXPERIMENTS

We evaluated the proposed system through a series of controlled experiments designed to assess the effectiveness of the integrated perception and control pipeline. Our experiments focused on three primary use cases, each of which is demonstrated in the accompanying video: (1) multi-person tracking with perception only, (2) single-person tracking with angular velocity control and (3) single-person tracking with lateral velocity control. The pipeline enables multi-object tracking but the control algorithm currently allows following a given single ID.

All components were deployed on the Triton robot running ROS Noetic inside a Docker container, due to the host operating system (Ubuntu 18.04) only supporting ROS Melodic (Python 2) natively. The Docker environment ensured compatibility with required packages while isolating the ROS workspace. The robot’s onboard camera produced 640×480 RGB images at 30 Hz. The perception stack (YOLOv3-tiny + SORT) achieved real-time performance at 10–15 FPS, with CPU utilization under 70% on the NVIDIA Jetson Nano. Control loops were executed at 30 Hz, with proportional gains and deadbands tuned empirically.

A. Multi-Person Tracking (Perception Only)

In the first segment of the demo, two individuals walk in and out of the robot’s field of view. The robot remains stationary, and only the YOLOv3-tiny + SORT tracker is active. This scenario evaluates the consistency of track ID assignment during entries, exits, and brief occlusions.

The tracker successfully maintains unique and persistent track IDs for both individuals across multiple appearances. Due to the integration of HSV histogram-based appearance features, the system correctly re-identifies previously tracked targets even after temporary disappearance, provided the visual features remain distinct. The HSV methods work better for simpler color schemes (like humans wearing single-coloured t-shirts/tops) which shows a limitation of such methods. These can be overcome using CNN-layer based features.

Failure cases typically occurred when occlusion duration exceeded the tracker’s max age or when two visually similar individuals crossed paths. It also happens when entry and exit points of objects change drastically due to lower assignment

TABLE I
PARAMETER VALUES IN THE PERCEPTION AND CONTROL PIPELINE

Parameter	Description	Default Value
confidence_threshold	Minimum objectness score for detection	0.5
nms_threshold	IoU threshold for non-max suppression	0.4
SORT.iou_threshold	IoU threshold for tracker-detection matching	0.3
SORT.feature_lambda	Weight for IoU vs. appearance feature cost	0.3
SORT.feature_threshold	Maximum Bhattacharyya distance allowed	0.7
SORT.max_age	Max frames without detection before removal	5
SORT.min_hits	Min detections before confirming track	10
k_p	Proportional gain for angular velocity control	0.25
k_v	Proportional gain for lateral velocity control	0.2
deadband	No-control region around image center (normalized)	0.03
control_rate	Control loop frequency	30 Hz
track_id	Target track ID to follow	(Default: first)
image_width	Image width for error normalization	640

TABLE II
ID ASSIGNMENT STABILITY DURING MULTI-PERSON DEMO

Person	Entries	Correct Re-IDs	ID Switches
A	3	3	0
B	3	3	0

of IoU cost in our re-identification algorithm. For this, we fine-tuned (empirically) parameter values of our pipeline to get good performance. Overall, the tracker demonstrated high ID consistency and robustness to short-term occlusion. Fig.4) shows the moments from the demo video for this experiment.

B. Single-Person Tracking with Angular Velocity Control

In the second segment, the robot actively controls its angular velocity to keep a single moving person centered in its field of view. The control policy is implemented in `bbox_center_follower.py`, using the horizontal offset of the bounding box center to generate a proportional angular velocity command.

The robot reacts promptly to lateral displacements of the target, rotating smoothly to re-center the bounding box within the defined deadband. Re-centering times averaged approximately 1.2 seconds for initial offsets of 25–35% of the image width. No oscillations or overshoots were observed under nominal conditions.

Rapid lateral motion of the person occasionally caused brief control lag due to detection jitter and hardware latency, though stability was maintained. The controller’s simplicity and responsiveness was effective for close-range object following.

C. Single-Person Tracking with Lateral Velocity Control

The final segment demonstrates control based purely on lateral (strafe) velocity, implemented in `bbox_center_strafe_follower.py`. In this mode, the robot uses sideways motion to reduce horizontal error without rotating its body, making it suitable for holonomic platforms.

Performance in this configuration closely matched the angular control setup. The robot consistently moved left or right to re-center the tracked person with an average convergence time of 1.3 seconds. Motion appeared more

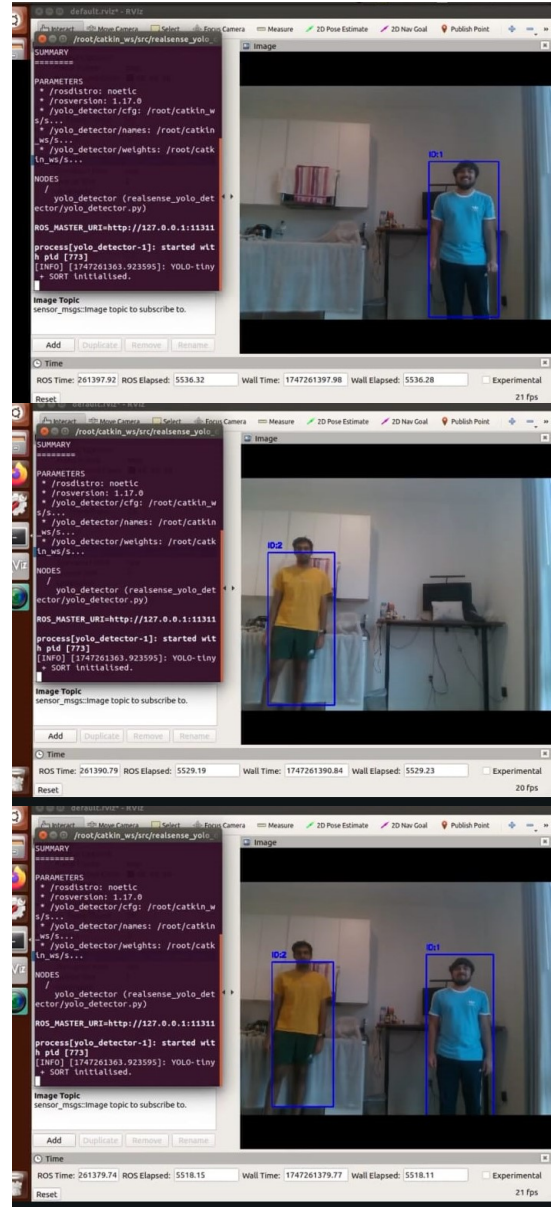


Fig. 4. Multi-object (human) tracking and re-identification

shaky than in angular control, and there was a visible jitter in the trajectory.

Limitations were observed when the tracked person moved outside the horizontal field of view, in which case the system could not reacquire the track without additional navigation strategies. We were planning on integrating mapping module for this but that is currently out of scope for the project.

TABLE III
SUMMARY OF TRACKING AND CONTROL PERFORMANCE

Metric	Perception Only	Angular	Lateral
Avg. ID consistency (ID switches/session)	<1	N/A	N/A
Avg. re-centering time (s)	N/A	1.2	1.3
Controller overshoot	N/A	Lower	Higher

The combined experiments confirm that the system provides real-time, robust tracking and control for visually distinct targets in moderately cluttered indoor scenes. Performance degrades in cases of ambiguous appearance or prolonged occlusion, and the lightweight computation makes it well-suited for embedded deployment on the Jetson Nano platform.

V. CONCLUSION AND FUTURE WORK

This project presented an integrated system for real-time object tracking and reactive (not proactive i.e. predictive) velocity control on the Triton robot using the SORT algorithm enhanced with appearance features and YOLOv3-tiny for object (human) detection. The perception pipeline combines efficient detection and robust tracking by leveraging Kalman filtering with color histogram-based data association, allowing consistent identity maintenance even under partial occlusions. The control subsystem implements two velocity-only modes: angular and lateral, that effectively align the robot's pose with the tracked target based solely on visual feedback.

Our experimental evaluation demonstrated that the system performs reliably in typical indoor environments on embedded hardware (NVIDIA Jetson Nano). The tracker maintained stable identities for multiple targets and successfully re-identified objects after short occlusions. Both control strategies produced smooth and timely re-centering behaviors, with average convergence times near one second. These results highlight the feasibility of lightweight, perception-driven reactive control for object following tasks without requiring explicit depth estimation or complex planning as long as they are short-term tasks. It was able to follow simple color-schemed objects due to HSV limitation, which motivated us to carry out experiments wearing single coloured/simple pattern t-shirts.

However, the system has limitations that motivate future work. The reliance on shallow HSV histograms for appearance features restricts robustness in scenarios with visually similar objects or rapidly changing lighting conditions. Incorporating more discriminative, learned embeddings from deep re-identification networks (layers from deep learning models like Convolutional Neural Network) could substantially improve identity preservation and reduce track fragmentation.

Additionally, extending control beyond velocity-only commands toward more predictive or model-based approaches may enhance smoothness and enable longer-term following in dynamic environments. Integrating the 2D-LiDAR or the depth map would also allow safer navigation and obstacle avoidance. We were currently limited by wire length due to screen terminal recording happening along with robot movements, so need to experiment in wireless mode conditions for larger distances.

Further development could explore multi-target tracking with coordinated control policies to follow multiple objects or switch targets dynamically. Finally, deploying and testing the system in outdoor or cluttered real-world scenarios would provide insights into generalization and robustness beyond the currently tested environment.

In conclusion, this project demonstrates that combining efficient detection, robust tracking with appearance cues, and simple velocity-based control forms a promising approach for practical robot object-following applications on resource-constrained platforms. It combines various fundamental topics taught throughout semester in CS 603 Robotics course. Architectures that facilitate improvements and extensions towards more autonomous and intelligent robotic systems compatible in edge-computing environments are directions for future research.

REFERENCES

- [1] Northrop Grumman, "MQ-4C Triton: ROS Model for Gazebo Simulation," <https://www.northropgrumman.com/Products/MQ4C-Triton/>, n.d., accessed: 2025-04-30.
- [2] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018. [Online]. Available: <https://arxiv.org/abs/1804.02767>
- [3] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *2016 IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.
- [4] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," *arXiv preprint arXiv:1703.07402*, 2017. [Online]. Available: <https://arxiv.org/abs/1703.07402>
- [5] G. Bradski, "The opencv library," *Dr. Dobbs's Journal of Software Tools*, 2000.
- [6] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

APPENDIX

TABLE IV
WORKLOAD ASSIGNMENT

Workload	Assigned To
Object Detection	Nilesh (33.33%), Pranav (33.33%), Sidisha (33.33%)
Object Tracking	Pranav (33.33%), Sidisha (33.33%), Kamalesh (33.33%)
Re-Identification	Nilesh (33.33%), Pranav (33.33%), Shantanu (33.33%)
Control and Navigation	Nilesh (33.33%), Kamalesh (33.33%), Shantanu (33.33%)
Integration	Nilesh (20%), Pranav (20%), Sidisha (20%), Shantanu (20%), Kamalesh (20%)
Testing	Nilesh (25%), Sidisha (25%), Shantanu (25%), Kamalesh (25%)